Tokenization for Noisy Texts: An Empirical Study on Named Entity Recognition

Alexander Malyy
Innopolis University | NLP (S26) | Case Study 1.7

Motivation & questions

Real-world text is noisy before any NLP model sees it: **OCR** errors, **ASR** transcripts without casing or punctuation, **social-media** abbreviations and typos. Tokenization is the first step of every modern pipeline; modern subword/byte-level tokenizers avoid OOV by construction — but do they stay *stable* under noise?

RQs: (1) How does noise perturb tokenization (fertility, token drift)? (2) Which tokenizer family (WordPiece / BPE / byte-level) is most noise-robust? (3) Can inference-time preprocessing or noisy fine-tuning recover NER F1, and which works when?

Experimental setup

Dataset: CoNLL-2003 English NER, 3,453 test sents, entity-level seqeval F1.

Models (tokenizer / vocab / F1 on clean):

bert-base-uncased	WordPiece / 30.5K	0.89
bert-base-cased	WordPiece / 28.9K	0.90
gpt2	BPE / 50.2K	0.69
byt5-small	byte / 256	0.86

Noise (p=0.3/tok): OCR visual confusions, ASR lowercase+phonetic, social typos+abbrev (Belinkov & Bisk, TextFlint).

Interventions: rule-based per-channel *preprocess* (word-count preserving) + mixed-noise *fine-tune* (1 epoch, 50% clean / 50% noise).

Key results (mean across 12 model-noise cells)

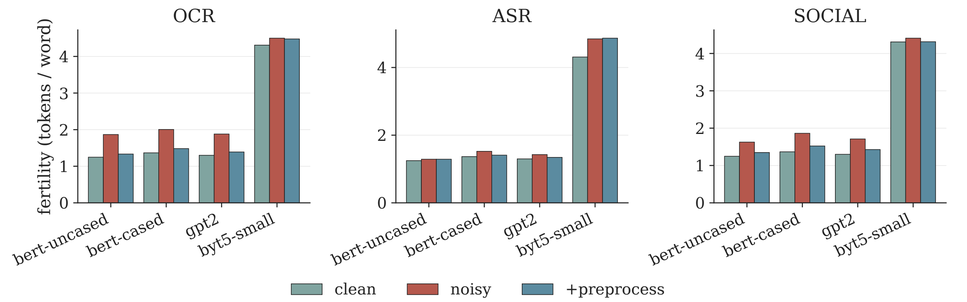
	mean F1	% recovery
Clean baseline	**0.83**	—
Noisy (no mitigation)	0.55 (−0.28)	0%
+ inference preprocess	0.69	49%
+ noisy fine-tune	0.69	47%
Best per cell	**0.71**	**55%**

Either intervention alone recovers ≈ half the noise gap; best-per-cell adds +0.02 over the winner. Neither fully closes the 0.28 drop.

Conclusions

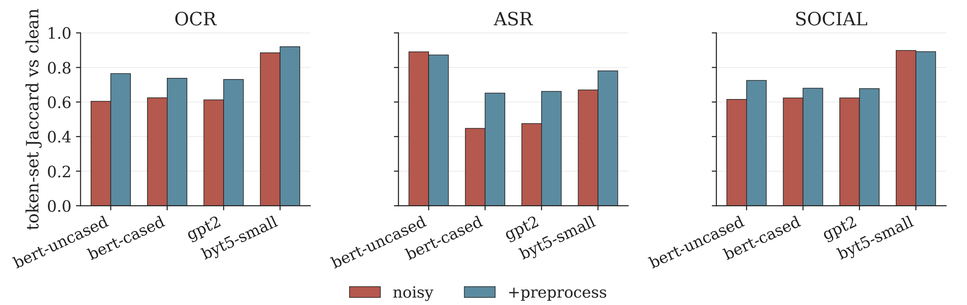
- "No UNK" ≠ noise-robust. Coverage = fertility inflation + token drift. Byte-level helps tokenizer layer, not F1 layer.
- Single component dominates per channel (truecase / ASR, charfix / OCR, spellcheck / social) — target, don't uniformly preprocess.
- Preprocess vs fine-tune = per-cell tradeoff; both act via **token stability** (causal lever, r=0.88).

Tokenizer fertility: clean → noisy → preprocess



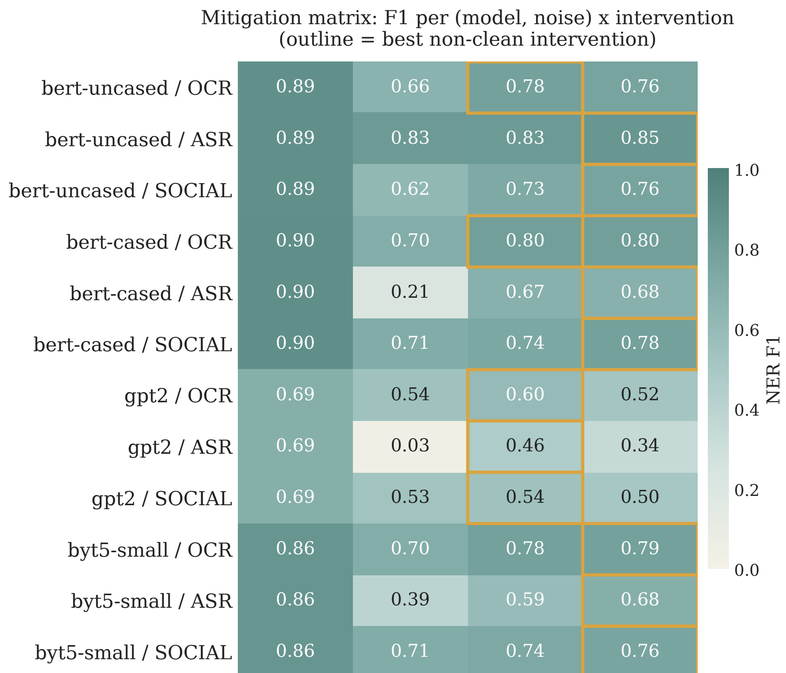
Subword tokenizers fragment heavily under OCR (**+45% to +50%** tokens/word). Byte-level is structurally noise-robust (**+2% social, +4% OCR, +12% ASR**). Preprocess restores fertility close to clean on OCR/social; ASR is modest (phonetic swaps produce in-vocab words).

Token-set stability: Jaccard vs clean reference



ASR scrambles *cased* tokenizer outputs most (Jaccard to **0.45-0.48**); truecasing recovers **half**. ByT5 stays ≥ 0.67 across all channels.

Mitigation matrix: F1 per (model, noise) × intervention

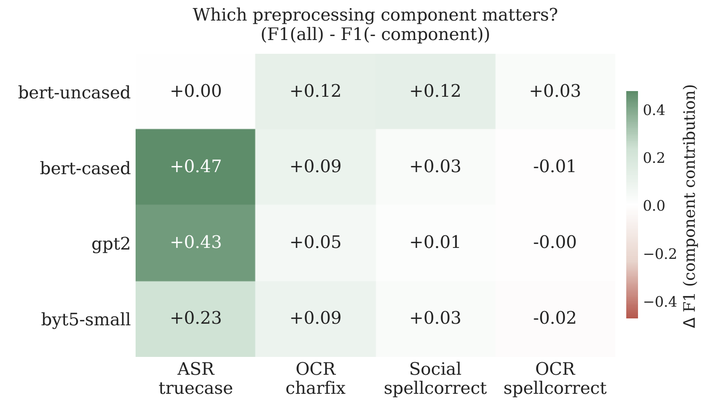


Preprocessing pipeline (per channel)

Channel	Pipeline	Example
OCR	char-fix → spellcheck	`hello` → `hello`
ASR	truecase → homophone-fix	`ur` → `your`
Social	un-repeat → un-abbrev → spellcheck	`gr8` → `great`

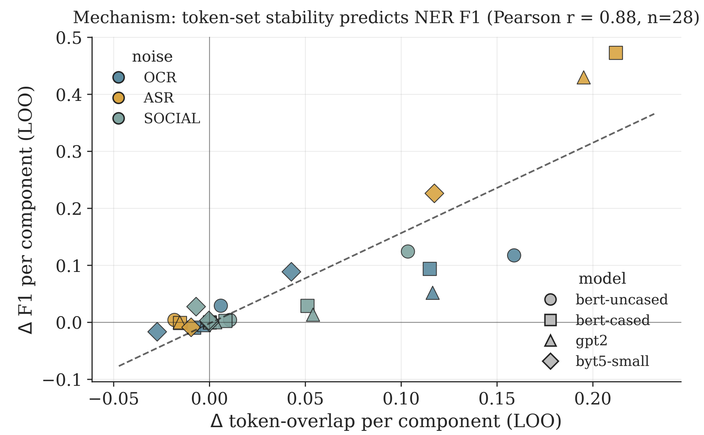
Word-count preserving — no splits, merges, or insertions. BIO labels stay aligned during NER eval. Spellcheck: pyspellchecker, edit-distance 1.

Which preprocessing component matters?



ASR **truecase** is life-saving for cased models (**+0.47** bert-cased, **+0.43** gpt2). OCR **charfix** dominates char-level recovery. OCR spell-correct **overcorrects** on cased/byte models (rewrites rare entities).

Mechanism: tokenization stability predicts F1



Across **28 LOO component cells**, F1 contribution is strongly predicted by token-set stability change (**Pearson** $r = 0.88$) — components that bring tokenization closer to clean recover NER F1.

Limitations & code

Limits: 1-epoch training; rule-based preprocess (no contextual MLM); punct-restore out-of-scope (BIO); English NER only; gpt2 clean F1 low (0.69) — absolute cross-model comparisons skew. Refs (Belinkov&Bisk, TextFlint, CoNLL, seqeval) in repo README.

Code: `github.com/n4rly-boop/S26-NLP-Tokenization-for-Noisy-Texts`